APOSTILA DE APOIO

Git e GitHub

Instalar o Git, criar a conta no GitHub, versionar projetos, trabalhar com branches e colaborar em grupo.

1. Introdução

Controle de versão é o registro das mudanças feitas em arquivos ao longo do tempo. Com ele, é possível voltar a versões anteriores, ver quem mudou o quê e trabalhar em equipe sem sobrescrever o trabalho de ninguém.

- **Git**: programa instalado no computador que controla as versões do projeto. Funciona offline.
- **GitHub**: site que hospeda repositórios Git na nuvem e permite compartilhar, revisar e colaborar.
- **Repositório**: pasta do projeto monitorada pelo Git.
- **Commit**: registro de mudanças, com uma mensagem descrevendo o que foi feito.
- **Branch**: linha paralela de desenvolvimento, usada para testar ideias sem afetar a versão principal.
- **Clone**: cópia de um repositório remoto para o computador.
- **Pull request**: pedido para unir as mudanças de uma branch à branch principal, após revisão.

2. Instalação e configuração

Instale o Git em cada computador do laboratório antes da aula. Depois, configure seu nome e e-mail, que aparecem em todos os commits.

1. Baixe o instalador em git-scm.com e aceite as opções padrão.
2. Abra o terminal (Prompt de Comando, PowerShell ou Git Bash) e confirme a instalação:

```
git --version
```

3. Configure sua identidade, usando o mesmo e-mail da conta GitHub:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

4. Confira as configurações:

```
git config --list
```

5. Defina o nome da branch principal como `main`:

```
git config --global init.defaultBranch main
```

3. Criando a conta no GitHub

1. Acesse github.com e clique em **Sign up**.
2. Informe e-mail, senha e um nome de usuário.
3. Confirme o e-mail recebido.
4. Escolha o plano gratuito.
5. Ative a autenticação em dois fatores em **Settings > Password and authentication**.

> **Importante:** o GitHub não aceita senha comum para operações pelo terminal. Use um **Personal Access Token** (em **Settings > Developer settings > Personal access tokens**) ou o login pelo navegador, via Git Credential Manager, instalado junto com o Git no Windows.

4. Comandos básicos

Os comandos abaixo são executados dentro da pasta do projeto.

Comando	Para que serve
`git init`	Cria um repositório Git na pasta atual
`git status`	Mostra arquivos modificados, novos e prontos para commit
`git add arquivo.txt`	Envia um arquivo para a área de preparo
`git add .`	Prepara todos os arquivos alterados
`git commit -m "mensagem"`	Registra as mudanças preparadas
`git log --oneline`	Lista o histórico de commits de forma resumida
`git diff`	Mostra as alterações ainda não preparadas
`git restore arquivo.txt`	Descarta alterações de um arquivo não commitado

Exemplo prático

```
mkdir meu-projeto
cd meu-projeto
git init
```

```
echo "Olá, Git" > README.md
git status
git add README.md
git commit -m "Primeiro commit"
git log --oneline
```

Ignorando arquivos

Crie um arquivo `.gitignore` para indicar o que o Git não deve registrar, como `node_modules/` ou senhas.

```
node_modules/
.env
*.log
```

5. Branches

Uma branch isola o trabalho de uma funcionalidade. A `main` deve permanecer estável.

Comando	Para que serve
`git branch`	Lista as branches locais
`git switch -c nova-tela`	Cria e troca para a branch
`git switch main`	Volta para a branch principal
`git merge nova-tela`	Une a branch `nova-tela` à branch atual
`git branch -d nova-tela`	Exclui uma branch já mesclada

Conflitos de merge

Se duas branches alteraram a mesma linha, o Git marca o trecho com `<<<<<<<`, `=======` e `>>>>>>>`. Edite o arquivo, escolha o conteúdo correto, remova as marcações e faça um novo commit.

6. Trabalho em grupo

O repositório principal fica no GitHub, o **remoto**, chamado `origin`. Cada integrante copia o projeto, trabalha na própria branch e envia as mudanças.

1. O líder cria o repositório e adiciona os colegas como **collaborators** (em **Settings > Collaborators**).
2. Cada aluno copia o repositório:

```
git clone https://github.com/grupo/projeto.git
cd projeto
```

3. Cada aluno cria a própria branch:

```
git switch -c feature-nome-do-aluno
```

4. Envia a branch pela primeira vez, e depois apenas `git push`:

```
git push -u origin feature-nome-do-aluno
git push
```

5. Para receber as mudanças dos colegas:

```
git pull origin main
```

Comando	Para que serve
`git remote -v`	Mostra o endereço do remoto
`git fetch`	Baixa as mudanças sem alterar seus arquivos
`git pull`	Baixa e une as mudanças
`git push`	Envia seus commits ao remoto

> **Regra de ouro:** rode `git pull` antes de começar a trabalhar e antes de enviar mudanças.

7. Pull request e compartilhamento

1. No GitHub, clique em **Compare & pull request**.
2. Escreva um título e uma descrição do que foi feito.
3. Indique um revisor e aguarde a aprovação.
4. Clique em **Merge pull request**.

Fork

Um **fork** é uma cópia do repositório de outra pessoa na sua conta. Use quando não tiver permissão de escrita no projeto original.

Resolvendo conflitos no pull request

Se o GitHub indicar conflito, atualize sua branch com a `main`:

```
git switch minha-branch
git pull origin main
# resolva os conflitos nos arquivos indicados
git add .
```

```
git commit -m "Resolve conflitos com main"
git push
```

> **Atenção:** nunca publique senhas, tokens ou arquivos `.env`.

8. Exercícios práticos

Faça os exercícios em duplas ou trios, na ordem.

☐ **Exercício 1:** instale o Git e rode `git --version`.

☐ **Exercício 2:** configure `user.name` e `user.email` e confira com `git config --list`.

☐ **Exercício 3:** crie uma conta no GitHub e ative a autenticação em dois fatores.

☐ **Exercício 4:** crie a pasta `meu-projeto`, rode `git init` e faça o primeiro commit com um `README.md`.

☐ **Exercício 5:** crie a branch `alteracao-01`, edite o README, faça um commit e volte para a `main`.

☐ **Exercício 6:** mescle a branch com `git merge alteracao-01` e apague-a com `git branch -d alteracao-01`.

☐ **Exercício 7:** o líder cria um repositório no GitHub e adiciona os colegas como colaboradores.

☐ **Exercício 8:** cada aluno clona o repositório, cria sua branch e envia com `git push -u origin`.

☐ **Exercício 9:** abra um pull request e peça revisão de um colega.

☐ **Exercício 10 (desafio):** provoque um conflito editando a mesma linha em duas branches e resolva-o.

Resumo dos comandos

Ação	Comando
Iniciar repositório	`git init`
Ver situação	`git status`

Preparar arquivos	`git add .`
Registrar mudanças	`git commit -m "mensagem"`
Ver histórico	`git log --oneline`
Criar e trocar branch	`git switch -c nome`
Unir branch	`git merge nome`
Copiar do GitHub	`git clone URL`
Enviar	`git push`
Receber	`git pull`